

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO
SUDESTE DE MINAS GERAIS - CAMPUS RIO POMBA**

STEPHANYE CRISTINE ANTUNES DE CUNTO

**ANÁLISE COMPARATIVA ENTRE THREADS TRADICIONAIS E VIRTUAIS NO
JAVA**

RIO POMBA - MG

2025

STEPHANYE CRISTINE ANTUNES DE CUNTO

**ANÁLISE COMPARATIVA ENTRE THREADS TRADICIONAIS E VIRTUAIS NO
JAVA**

Trabalho de Conclusão de curso apresentado ao *Campus* Rio Pomba, do Instituto Federal de Educação, Ciência e Tecnologia do Sudeste de Minas Gerais, como parte das exigências do curso de Bacharelado em Ciência da Computação para a obtenção do título de Bacharel em Ciência da Computação.

Orientadora: ME. BIANCA PORTES DE CASTRO

Coorientador: DR. JOSÉ RUI CASTRO DE SOUSA

RIO POMBA - MG

2025

FICHA CATALOGRÁFICA TEMPORÁRIA
STEPHANYE CRISTINE ANTUNES DE CUNTO
ANÁLISE COMPARATIVA ENTRE THREADS TRADICIONAIS E VIR-
TUAIS NO JAVA/ STEPHANYE CRISTINE ANTUNES DE CUNTO. – RIO
POMBA - MG, 2025-

Orientadora: ME. BIANCA PORTES DE CASTRO

Trabalho de Conclusão de Curso – Instituto Federal de Educação,
Ciência e Tecnologia do Sudeste de Minas, Campus Rio Pomba

STEPHANYE CRISTINE ANTUNES DE CUNTO

**ANÁLISE COMPARATIVA ENTRE THREADS TRADICIONAIS E VIRTUAIS NO
JAVA**

Trabalho de Conclusão de curso apresentado ao *Campus* Rio Pomba, do Instituto Federal de Educação, Ciência e Tecnologia do Sudeste de Minas Gerais, como parte das exigências do curso de Bacharelado em Ciência da Computação para a obtenção do título de Bacharel em Ciência da Computação.

Trabalho aprovado em XX de XXXXX de XXXX.

ME. BIANCA PORTES DE CASTRO
Orientadora, IF Sudeste MG - Rio Pomba

DR. JOSÉ RUI CASTRO DE SOUSA
Coorientador, IF Sudeste MG - Rio Pomba

TÍTULO E NOME DO MEMBRO DA BANCA
IF Sudeste MG - Rio Pomba

RIO POMBA - MG

2025

*“Eu tenho uma porção de coisas grandes pra conquistar,
e eu não posso ficar aí parado.”
(Raul Seixas)*

Resumo

As *threads* virtuais, introduzidas a partir do Java 19, diferenciam-se das *threads* tradicionais por serem gerenciadas diretamente pela Java Virtual Machine, o que reduz a demanda por recursos computacionais durante a execução das aplicações. Ainda existem poucos estudos que avaliam sua eficiência e comportamento. Diante desse contexto, o presente trabalho teve como objetivo analisar o desempenho das *threads* virtuais em comparação às *threads* tradicionais, considerando aspectos relacionados à utilização de recursos e ao comportamento da aplicação sob diferentes condições de execução. A pesquisa concentrou-se em atividades de *I/O*, caracterizadas por períodos frequentes de espera, cenário em que se espera que as *threads* virtuais proporcionem melhor aproveitamento dos recursos computacionais. Para isso, foram realizados testes em ambiente controlado, permitindo a observação e comparação dos resultados obtidos com cada modelo de thread. Como resultado, observou-se que as *threads* virtuais apresentam uso de *heap* e memória semelhante ao das *threads* tradicionais, embora apresentem uso de CPU menor. Este estudo contribui para o entendimento do funcionamento das *threads* virtuais e das principais diferenças observadas em comparação às *threads* tradicionais.

Palavras-chave: *Threads* virtuais. Java. Concorrência. Desempenho. *Threads* tradicionais.

Abstract

Virtual threads, introduced in Java 19, differ from traditional threads in that they are managed directly by the Java Virtual Machine, which reduces the demand for computational resources during application execution. There are still few studies evaluating their efficiency and behavior. In this context, the present work aimed to analyze the performance of virtual threads compared to traditional threads, considering aspects related to resource usage and application behavior under different execution conditions. The research focused on I/O-intensive activities, characterized by frequent waiting periods, a scenario in which virtual threads are expected to provide better utilization of computational resources. To this end, tests were conducted in a controlled environment, allowing the observation and comparison of results obtained with each thread model. As a result, it was observed that virtual threads exhibit heap and memory usage similar to that of traditional threads, while presenting lower CPU usage. This study contributes to the understanding of the functioning of virtual threads and the main differences observed in comparison to traditional threads.

Key-words: Virtual threads. Java. Concurrency. Performance. Traditional threads.

Lista de ilustrações

Figura 1	–	Comparação entre processo com <i>single thread</i> e múltiplas <i>threads</i>	15
Figura 2	–	Modelo 1:1 (um-para-um) de mapeamento de <i>threads</i> de usuário para <i>threads</i> de <i>kernel</i>	16
Figura 3	–	Arquitetura de <i>threads</i> virtuais: múltiplas <i>threads</i> virtuais multiplexadas sobre <i>carrier threads</i>	18

Lista de tabelas

Tabela 1	– Redução estimada da margem de erro do <i>rate</i> (<i>threads</i> tradicionais), erro relativo e tempo total de execução.	26
Tabela 2	– Média da latência sob carga constante	32
Tabela 3	– Utilização média de recursos sob carga constante	33
Tabela 4	– Taxa média de requisições (<i>rate</i>) sob carga gradual	34
Tabela 5	– Utilização média de recursos sob carga gradual	34

Lista de abreviaturas e siglas

API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
AWS	<i>Amazon WEB Services</i> (Serviços WEB da Amazon)
CPU	Unidade Central de Processamento
EC2	<i>Elastic Compute Cloud</i> (Computação em Nuvem Elástica)
HTTP	<i>Hypertext Transfer Protocol</i> (Protocolo de Transferência de Hipertexto)
I/O	Entrada/Saída
IQR	<i>Interquartile Range</i> (Intervalo Interquartil)
JDK	<i>Java Development Kit</i> (Kit de Desenvolvimento Java)
JEP	<i>Java Enhancement Proposal</i> (Proposta de Aprimoramento do Java)
JFR	<i>Java Flight Recorder</i> (Gravador de Voo Java)
JMC	<i>Java Mission Control</i> (Controle de Missão Java)
JMH	<i>Java Microbenchmark Harness</i>
JSON	<i>JavaScript Object Notation</i> (Notação de Objetos JavaScript)
JVM	<i>Java Virtual Machine</i> (Máquina Virtual Java)
LTS	<i>Long Term Support</i> (Suporte de Longo Prazo)
MB	<i>Megabyte</i>
RAM	<i>Random Access Memory</i> (Memória de Acesso Aleatório)
REST	<i>Representational State Transfer</i>
SO	Sistema Operacional
SPD	Sistemas Paralelos e Distribuídos
TCP	<i>Transmission Control Protocol</i> (Protocolo de Controle de Transmissão)
Wi-Fi	<i>Wireless Fidelity</i> (Fidelidade Sem Fio)

Sumário

1	Introdução	12
2	Fundamentação Teórica	14
2.1	Processos e Threads	14
2.2	Threads Tradicionais em Java	15
2.3	Limitações das Threads Tradicionais	16
2.4	Threads Virtuais (Project Loom)	17
3	Trabalhos Relacionados	19
3.1	Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads	19
3.2	Uma análise comparativa entre threads e green threads no Java	19
3.3	Comparison of Concurrency Technologies in Java	20
3.4	Avaliação dos mecanismos de concorrência na API do Java 8	20
3.5	Síntese dos Trabalhos Relacionados	21
4	Metodologia	22
4.1	Método de Pesquisa	22
4.2	Ambiente de Teste	22
4.3	Implementação do Servidor	24
4.4	Cenários avaliados	25
4.4.1	Cenário 1: cenário de criação de <i>threads</i> de forma constante	27
4.4.2	Cenário 2: cenário de criação de <i>threads</i> com aumento gradual (<i>ramp up</i>)	28
4.5	Métricas Coletadas	29
4.6	Tratamento estatísticos dos dados	29
4.7	Tratamento de erros	30
4.8	Limitações dos Testes	31
5	Resultados e Discussão	32
5.1	Resultados obtidos referentes ao Cenário 1 - carga constante:	32
5.2	Resultados obtidos referentes ao Cenário 1 - carga gradual (<i>Ramp Up</i>)	34
5.2.1	Implicações Práticas	36
6	Conclusão	37
	Referências	39

1 Introdução

Uma *thread* é a menor unidade de processamento existente dentro de um processo. Cada processo é capaz de conter múltiplas *threads*, permitindo a execução simultânea de diferentes partes de um programa ou de diferentes tarefas (Silberschatz; Galvin; Gagne, 2018).

A utilização de *threads* oferece diversos benefícios em problemas que envolvem tarefas de alto processamento (*CPU-bound*) e operações de entrada e saída (*I/O-bound*). Além disso, em sistemas reativos e servidores, *threads* permitem que aplicações atendam múltiplos usuários simultaneamente, mantendo respostas rápidas e contínuas mesmo sob alta carga, garantindo desempenho eficiente e melhor experiência ao usuário. Assim, o uso de *threads* é aplicado como estratégia para otimizar recursos computacionais e melhorar a eficiência de sistemas em diferentes contextos.

O relatório *State of the Octoverse 2024* (GitHub, 2024) demonstra que a linguagem de programação Java está entre as cinco linguagens mais utilizadas na plataforma, o que reforça sua importância como uma das principais tecnologias do desenvolvimento de *software*.

No lançamento da versão 19 do Java, foram introduzidas as *threads* virtuais, que, diferente das *threads* tradicionais, são gerenciadas pela *Java Virtual Machine* (JVM). Enquanto o escalonamento das *threads* tradicionais é realizado pelo sistema operacional (SO), determinando quando cada *thread* é executada, o escalonamento das *threads* virtuais é feito pela própria JVM, podendo apresentar comportamentos distintos (Oracle Corporation, 2023).

O uso de *threads* tradicionais pode gerar *overhead* e limitar a escalabilidade. Dessa forma, as *threads* virtuais surgem como alternativa para contornar essas limitações. Este trabalho investiga como a utilização de *threads* virtuais impacta o desempenho e a escalabilidade de aplicações concorrentes.

O objetivo deste trabalho é analisar as diferenças de desempenho entre *threads* tradicionais e *threads* virtuais, avaliando como cada abordagem impacta a execução de aplicações concorrentes.

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Compreender os fundamentos teóricos da programação concorrente e das diferentes abordagens de *threads*;
- Medir e comparar métricas de desempenho;
- Analisar os resultados obtidos, identificando vantagens e limitações.

A escolha deste tema parte do interesse gerado durante a disciplina de Sistemas Paralelos e Distribuídos (SPD), na qual foram estudados os conceitos fundamentais de concorrência, paralelismo e gerenciamento de *threads*. Além disso, o apreço pessoal pela linguagem Java contribuiu para a definição do tema, considerando que a plataforma tem investido em melhorias relacionadas à programação concorrente. Dessa forma, esse trabalho se justifica pelo interesse acadêmico adquirido ao longo da disciplina e pela pertinência de investigar uma inovação recente da linguagem Java, contribuindo para a compreensão de como esse novo modelo de *threads* pode aprimorar o desempenho de aplicações concorrentes.

Este trabalho está organizado da seguinte forma:

- **Seção 1 - Introdução:** Apresenta o tema, os objetivos e a justificativa.
- **Seção 2 - Fundamentação teórica:** Expõe os conceitos de concorrência e paralelismo e descreve o funcionamento de processos, *threads* tradicionais e *threads* virtuais.
- **Seção 3 - Trabalhos Relacionados:** Revisa estudos anteriores sobre o desempenho de *threads* tradicionais e *threads* virtuais.
- **Seção 4 - Metodologia:** Detalha como os testes foram conduzidos e como os dados foram coletados.
- **Seção 5 - Resultados e Discussão:** Apresenta e analisa os dados obtidos durante os testes.
- **Seção 6 - Conclusão:** Expõe as considerações finais do estudo.

Por fim, são listadas as referências bibliográficas utilizadas no trabalho.

2 Fundamentação Teórica

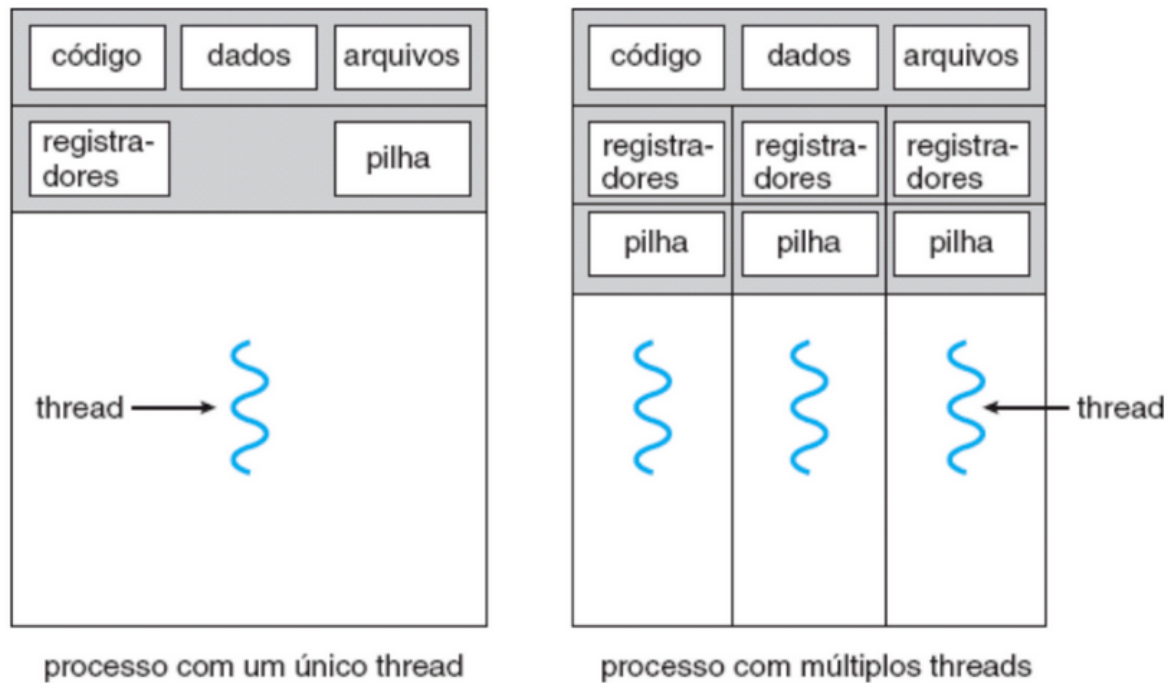
Esta seção apresenta os conceitos e teorias necessários para a compreensão do trabalho, fornecendo embasamento científico às análises e às decisões adotadas ao longo deste estudo. São abordados fundamentos de sistemas operacionais, como processos, *threads* e concorrência.

2.1 Processos e Threads

Segundo Silberschatz, Galvin e Gagne (2018), um processo pode ser definido como um programa em execução, não se limitando apenas ao código do programa. O conceito de processo inclui também o seu estado atual durante a execução no sistema operacional, abrangendo informações como o contador de programa, registradores da CPU e áreas de memória próprias, como pilha, área de dados e *heap*. Além disso os processos podem possuir estados diferentes ao longo do seu ciclo de vida, como criação, execução, espera e término.

Uma *thread* pode ser definida como a unidade básica de utilização da CPU, sendo composta por um identificador (*thread ID*), um contador de programa, um conjunto de registradores e uma pilha. *Threads* pertencentes ao mesmo processo podem compartilhar a seção de código, a seção de dados e os recursos do sistema operacional. Todo processo possui ao menos uma *thread* de controle, quando contém múltiplas *threads*, torna-se capaz de executar mais de uma tarefa de forma concorrente. (Silberschatz; Galvin; Gagne, 2018)

De acordo com Silberschatz, Galvin e Gagne (2018), o uso de múltiplas *threads* apresenta diversos benefícios, como o aumento da capacidade de reposta, uma vez que o uso de *threads* permite que o programa continue em execução mesmo quando parte dele está bloqueado, aumentando a interatividade com o usuário. Outro benefício é o compartilhamento de recursos, pois, diferente dos processos nos quais a comunicação ocorre por meio de memória compartilhada ou troca de mensagens, *threads* permitem o compartilhamento direto de recursos dentro do mesmo espaço de endereçamento. Além disso, o uso de *threads* proporciona economia de recursos, uma vez que a criação e o gerenciamento de *threads* demandam menor custo de memória quando comparados aos processos. Em arquiteturas multiprocessadas, *threads* podem ser executadas em paralelo em diferentes núcleos de processamento, enquanto um processo com apenas uma *thread* é limitado à execução em um único núcleo.

Figura 1 – Comparação entre processo com *single thread* e múltiplas *threads*.

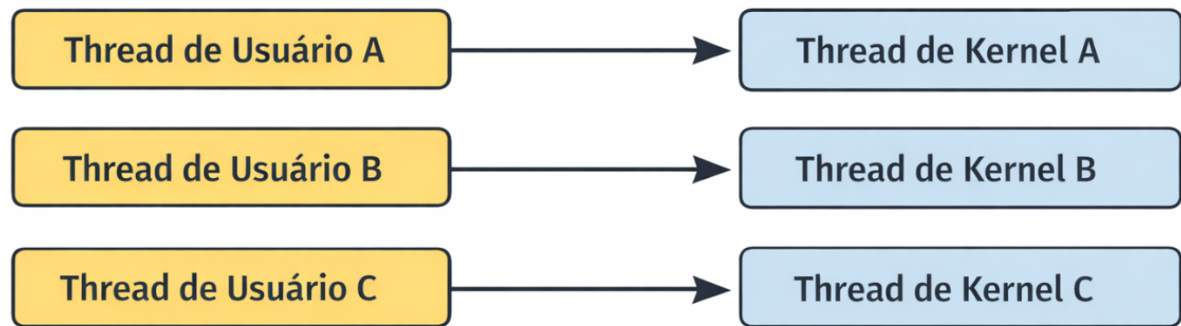
Fonte: Silberschatz, Galvin e Gagne (2018).

A Figura 1 ilustra as diferenças entre processos de *thread* única e múltiplas *threads*. Enquanto um processo com uma única *thread* possui apenas um conjunto de registradores e uma pilha, a utilização de múltiplas *threads* permite que cada *thread* mantenha seu próprio contexto de execução, compartilhando código e dados do processo. Essa comparação ajuda a compreender os benefícios de desempenho e flexibilidade trazidos pelo modelo *multithread*.

2.2 Threads Tradicionais em Java

Threads tradicionais em Java seguem o modelo de mapeamento *one-to-one* (1:1), no qual cada *thread* de nível de usuário criada pelo aplicativo é mapeada para uma *thread* de nível de *kernel*. Com isso, todas as *threads* podem acessar o *kernel* simultaneamente e serem agendadas de forma independente. Esse modelo de mapeamento torna-se limitado, cada *thread* adicional aumenta o peso do processo, consumindo mais recursos do sistema. Consequentemente, muitas implementações suportam um número limitado de *threads* simultâneas, imposto pelo sistema operacional (Oracle Corporation, 2010).

Figura 2 – Modelo 1:1 (um-para-um) de mapeamento de *threads* de usuário para *threads* de *kernel*.



Fonte: Elaborado pela autora (2025).

A Figura 2 ilustra o modelo 1:1 de *threads*, onde cada *thread* de usuário é associada diretamente a uma *thread* de *kernel*.

O Java fornece uma API básica para programação concorrente por meio da classe `java.lang.Thread`, permitindo que a JVM gerencie múltiplas *threads* de forma simultânea. A interface `Runnable` define a unidade de execução que pode ser associada a uma *thread*. Dessa forma, uma *thread* pode ser criada tanto pela instanciação direta da classe `Thread` quanto pelo fornecimento de um objeto que implemente `Runnable` ao seu construtor. A execução de uma *thread* é iniciada pela chamada do método `start()`, que delega à JVM a invocação assíncrona do método `run()`. Durante seu ciclo de vida, uma *thread* pode transitar entre diferentes estados `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING` e `TERMINATED` permitindo ao programador compreender e controlar seu comportamento ao longo da execução. (Oracle Corporation, 2014b)

2.3 Limitações das Threads Tradicionais

A documentação oficial da Oracle (Oracle Corporation, 2014a) define que por padrão o tamanho da pilha gerada por *thread* é aproximadamente 1 MB. Com a criação de milhares de *threads*, o consumo total de memória se torna significativo, por exemplo a criação de 10.000 *threads* de plataforma resultaria em cerca de 10 GB de memória ocupada apenas pelas pilhas, sem contar outros recursos associados a cada *thread*.

Há também o custo de criação que envolve interações diretas com o sistema operacional, o que exige chamadas de sistema (*system calls*) e a alocação de estruturas de gerenciamento de *threads* no *kernel*. Gerando assim um *overhead* maior do que a criação de objetos comuns na JVM. Ao utilizar muitas *threads* esse custo pode se tornar elevado e impactar na performance do sistema (Oracle Corporation, 2014a).

A troca de contexto (*context switch*) também é um fator importante. De acordo com Tanenbaum e Bos (2015), o sistema operacional deve salvar o estado da *thread* atual

(registradores, ponteiro de instrução, pilha, *caches*, etc.) e restaurar o estado da *thread* que será executada em seguida. Esse processo gera *overhead*, e seu custo impacta diretamente o desempenho do sistema, contribuindo para tornar o modelo tradicional de *threads* (1:1) menos eficiente em cenários com milhares de *threads*.

O uso de *threads* tradicionais pode ser limitado em termos de escalabilidade, uma vez que o sistema operacional impõe restrições ao número de *threads* que podem ser criadas simultaneamente. No modelo *thread-per-request* onde cada tarefa cria sua própria *thread*, a abordagem torna-se ineficiente em cenários de alta concorrência, não apenas pelo consumo de memória, mas também pelo *overhead* de criação e custo de troca de contexto. Gerando o problema conhecido como C10K, que descreve a dificuldade de gerenciar mais de 10.000 conexões simultâneas utilizando *threads* tradicionais. (Kegel, 2006)

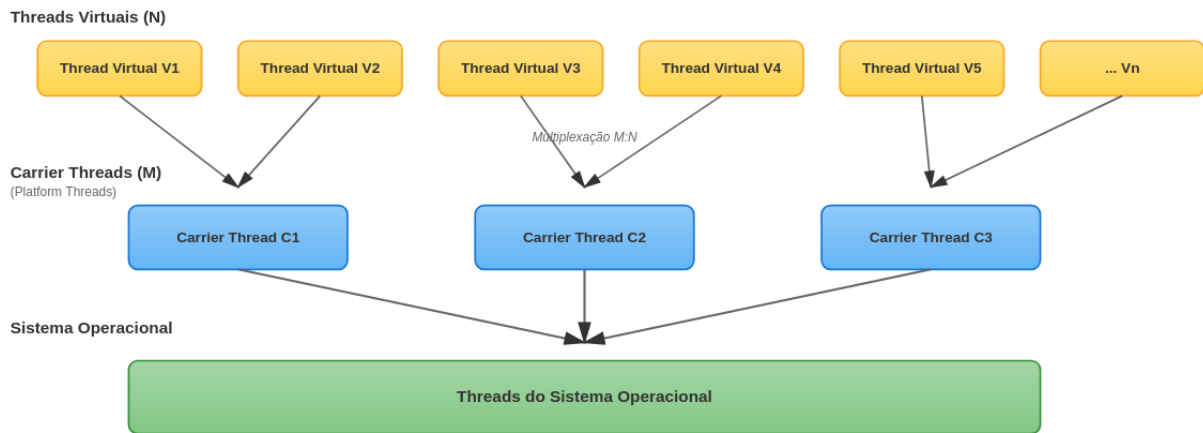
Como cada *thread* de plataforma é gerenciada pelo sistema operacional, todas competem por recursos limitados como estruturas de controle, tempo de CPU e *caches* do processador. Enquanto o número de *threads* aumenta, o *overhead* de escalonamento (*scheduling overhead*) também cresce, reduzindo a eficiência do sistema (Tanenbaum; Bos, 2015).

2.4 Threads Virtuais (Project Loom)

O *Project Loom* foi iniciado no final de 2017, com foco em melhorar a escalabilidade de sistemas *WEB* e aplicações concorrentes através do paralelismo. O modelo de *threads* tradicionais não escala bem em cenários com um grande número de tarefas simultâneas, para contornar essas limitações o projeto apresentou um novo modelo de *thread*, as *threads* virtuais. (OpenJDK Project Loom, 2020)

De acordo com a documentação do *Project Loom* OpenJDK Project Loom (2020) *threads* virtuais são *threads* leves gerenciadas pela JVM, que não estão diretamente vinculadas a *threads* do sistema operacional. Na API do Java, elas se apresentam como *threads* normais, permitindo que o programador interaja com elas de forma muito semelhante ao uso de *threads* tradicionais. A proposta visa melhorar a eficiência de criação de milhares de *threads* consumindo menos memória e evitando *overhead* de criação e troca de contexto.

Enquanto as *threads* tradicionais utilizam o modelo 1:1, segundo a documentação oficial da Oracle Oracle Corporation (2023), as *threads* virtuais são executadas sobre um conjunto menor de *platform threads* ou *carrier threads*. Isso permite que uma grande quantidade de *threads* virtuais seja multiplexada sobre um número limitado de *threads* do sistema operacional, mecanismo análogo ao modelo M:N de multiplexação, no qual muitas *threads* lógicas (N) são associadas a menos *threads* físicas (M), melhorando assim a escalabilidade do sistema.

Figura 3 – Arquitetura de *threads* virtuais: múltiplas *threads* virtuais multiplexadas sobre *carrier threads*

Fonte: Elaborado pela autora (2025).

A Figura 3 ilustra o funcionamento das *threads* virtuais, mostrando como múltiplas *threads* lógicas podem ser multiplexadas sobre um número menor de *carrier threads*, de forma análoga ao modelo M:N.

Há dois conceitos, *mounting* e *unmounting*, que descrevem como uma *thread* virtual é associada a uma *carrier thread* para executar suas instruções. Quando a *thread* encontra uma operação bloqueante, como operações de I/O (*sockets*, arquivos), `Thread.sleep()` ou `Object.wait()`, ela é desmontada da *carrier thread*, liberando esta última para executar outras *threads* virtuais. Após a conclusão da operação bloqueante, a *thread* virtual é remontada em uma *carrier thread* disponível, retomando sua execução. Assim esse mecanismo permite a execução de um grande número de *threads* virtuais sobre um número limitado de *carrier threads*, aumentando a escalabilidade e reduzindo o consumo de recursos do sistemas Oracle Corporation (2023).

Na documentação Oracle Corporation (2014b) também é descrito que as *threads* virtuais possuem sua pilha (*stack*) alocada dinamicamente no *heap* da JVM, enquanto as *threads* tradicionais possuem sua pilha fixa e gerenciada pelo sistema operacional. Essa pilha dinâmica cresce e reduz conforme a necessidade, com tamanho inicial muito menor aproximadamente 1 KB.

Podemos observar que *threads* virtuais são adequadas para *workloads I/O-bound*, onde a maior parte do tempo é gasta aguardando operações de entrada/saída. Em tarefas *CPU-bound*, o benefício é limitado, já que o gargalo ainda será o número de *cores* disponíveis.

3 Trabalhos Relacionados

Esta seção apresenta trabalhos que servem de base para a pesquisa sobre *threads* tradicionais e *threads* virtuais em Java. São discutidos estudos anteriores que investigaram o desempenho dessas *threads*. Com a revisão realizada, é possível compreender as abordagens utilizadas, os resultados obtidos e as lacunas existentes, fornecendo um referencial teórico sólido para os testes conduzidos neste trabalho.

3.1 Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads

Pandita (2024) apresentou uma comparação entre *threads* virtuais e *threads* tradicionais, considerando escalabilidade, utilização de recursos e latência em cargas de trabalho de alto rendimento. Foram realizados dois *benchmarks*: *CPU-bound*, envolvendo cálculos de números primos, e *I/O-bound*, simulando operações de bloqueio entre duas aplicações Java em ambiente AWS EC2.

Os resultados mostraram que, em tarefas com baixo uso de CPU, ambos os modelos apresentam desempenho semelhante. Em cenários intensivos de CPU, a limitação dos recursos computacionais restringe o aumento da simultaneidade. Por outro lado, em operações *I/O-bound*, *threads* virtuais se destacaram, oferecendo maior simultaneidade, redução do consumo de memória e melhor aproveitamento da CPU. Esses dados reforçam a eficiência das *threads* virtuais em operações de I/O e fornecem um referencial relevante para estudos comparativos, como o presente trabalho.

3.2 Uma análise comparativa entre threads e green threads no Java

Souto (2024) realizou uma pesquisa comparando o desempenho de *threads* tradicionais e *threads* virtuais. O estudo analisou operações fundamentais de gerenciamento de *threads*, o tempo necessário para instanciá-las, iniciá-las, finalizá-las e realizar a troca de contexto em ambas. Para isso, foram utilizados testes com diferentes quantidades de *threads* e a biblioteca JMH do JDK para coleta precisa de tempos de execução.

O estudo demonstrou que, nos cenários testados, as *threads* virtuais podem ser mais de 100 vezes mais rápidas que as *threads* tradicionais, evidenciando sua eficiência em operações de criação, inicialização, finalização e mudança de contexto. Apesar do alto desempenho observado, o estudo se concentrou em cenários específicos, abrindo espaço para pesquisas adicionais que explorem diferentes cargas de trabalho e padrões de uso de *threads*, como os apresentados neste trabalho.

3.3 Comparison of Concurrency Technologies in Java

Em seu trabalho, Gustafsson e Persson (2024) realizou quatro *benchmarks* com o objetivo de comparar o desempenho entre *threads* tradicionais, *threads* virtuais e *Reactive Framework* em testes de *I/O-bound* (simulado), *CPU-bound* e testes mistos.

No teste *I/O-bound*, utilizou-se o mecanismo `Thread.sleep(100 ms)` para simular tempos de espera. No teste *CPU-bound*, realizou-se a multiplicação de matrizes 200x200. Os dois testes mistos combinaram cálculos de matrizes e pausas: um focado em *I/O-bound* multiplicando matrizes 50x50 com pausa de 100 ms, e outro focado em *CPU-bound* multiplicando matrizes 200x200 com pausa de 50 ms.

Os resultados indicaram que, no teste *CPU-bound*, *threads* tradicionais apresentaram melhor desempenho em termos de *throughput* e latência. No teste *I/O-bound*, o *Reactive Framework* se destacou, mostrando maior escalabilidade e menor uso de CPU e memória. Nos testes mistos, *threads* virtuais obtiveram alta taxa de requisições e baixa latência no cenário *CPU-bound*, mantendo desempenho elevado no cenário *I/O-bound*, enquanto *threads* tradicionais apresentaram maior latência e menor *throughput*. O que torna notório que a eficácia das *threads* virtuais em cenários mistos, evidenciando seu potencial em aplicações que demandam simultaneidade e eficiência de recursos.

3.4 Avaliação dos mecanismos de concorrência na API do Java 8

Uma análise comparativa do desempenho de *single thread*, *threads*, *ExecutorService* e *Fork/Join* em algoritmos de ordenação foi realizada por Águas (2015), utilizando os algoritmos *Quicksort*, *Merge-sort* e *Pigeonhole Sort*. Os testes foram conduzidos em três máquinas com processadores diferentes (*Dual-Core*, i5 e i7), e cada algoritmo foi executado 60 vezes para cada abordagem de concorrência.

O *Quicksort* executado com *ForkJoin-Pool* em uma máquina com 8 processadores lógicos apresentou o melhor desempenho em termos de tempo. De forma geral, *Quicksort* combinado com *ForkJoin-Pool* registrou os menores tempos na maioria dos cenários testados. O estudo demonstrou que a máquina com maior número de processadores lógicos obteve os melhores resultados, comprovando que a execução paralela se beneficia diretamente da disponibilidade de mais núcleos para distribuir as tarefas simultaneamente.

Esse trabalho evidencia a importância da escolha adequada de técnicas de concorrência e da arquitetura de *hardware* para otimização do desempenho em algoritmos paralelos.

3.5 Síntese dos Trabalhos Relacionados

Em seu texto Souto (2024) concentrou-se na análise das operações de criação, inicialização, finalização e troca de contexto, onde se nota que *threads* virtuais podem ser mais de 100 vezes mais rápidas que *threads* tradicionais em cenários específicos. Por sua vez Pandita (2024) aprofunda essa análise ao avaliar o desempenho em cargas *CPU-bound* e *I/O-bound*, onde nota-se que, em tarefas com baixo uso de CPU, o desempenho é semelhante, porém *threads* virtuais se destacam em operações I/O, oferecendo maior simultaneidade, redução do consumo de memória e melhor aproveitamento da CPU.

Gustafsson e Persson (2024) expandiu a análise incluindo o *Reactive Framework* e cenários mistos, mostrando que *threads* virtuais apresentam alta taxa de requisições e baixa latência em cenários combinados de CPU e I/O, enquanto *threads* tradicionais apresentam limitações de *throughput* e latência. Enquanto Águas (2015) avaliou abordagens de concorrência na execução de algoritmos de ordenação, deixando claro que a execução paralela se beneficia diretamente do aumento do número de núcleos, principalmente quando combinada com *ForkJoin-Pool*, o que demonstra a importância do alinhamento entre técnicas de concorrência e arquitetura de *hardware*.

A comparação desses estudos permite identificar padrões: *threads* virtuais oferecem vantagens em simultaneidade e eficiência de recursos, especialmente em operações de I/O ou cenários mistos, enquanto *threads* tradicionais ainda podem apresentar desempenho competitivo em tarefas *CPU-bound* simples.

O presente trabalho se apoia nesses estudos como referencial para analisar o desempenho de *threads* tradicionais e virtuais em Java em operações de I/O, destacando diferenças de comportamento em operações básicas de criação, execução e finalização de *threads*. Dessa forma, busca-se fornecer uma compreensão prática sobre a eficiência e o uso de recursos dessas duas abordagens de concorrência em cenários controlados.

4 Metodologia

4.1 Método de Pesquisa

Este estudo tem uma abordagem quantitativa, de natureza aplicada e caráter experimental, tendo como objetivo comparar o desempenho entre threads tradicionais e threads virtuais em Java. Os experimentos foram baseados no trabalho de (Gustafsson; Persson, 2024), com adaptações realizadas para adequação ao contexto e ambiente deste estudo.

Essa seção descreve a metodologia adotada para comparar o desempenho entre *threads* tradicionais e *threads* virtuais em Java. Os experimentos foram baseados no trabalho de Gustafsson e Persson (2024), como adaptações para o contexto e ambiente deste estudo.

A abordagem envolveu a implementação de um servidor HTTP simples utilizando arquitetura cliente-servidor, onde o servidor foi configurado para utilizar *threads* tradicionais e *threads* virtuais. Dois tipos de testes foram conduzidos: testes de carga constante para avaliar estabilidade ao longo do tempo, e testes de aumento gradual para identificar limites de escalabilidade.

Todo o código-fonte, *scripts* de automação e configurações utilizadas neste estudo estão disponíveis publicamente no repositório *GitHub*¹, permitindo a replicação completa dos experimentos.

As subseções seguintes detalham as escolhas feitas para a execução dos experimentos.

4.2 Ambiente de Teste

Os testes utilizaram uma arquitetura cliente-servidor, como duas máquinas conectadas por uma rede *Wi-fi* local. A máquina servidora era responsável por hospedar a aplicação Java e coletar métricas de desempenho, enquanto a máquina cliente era utilizada para gerar carga e coletar métricas de latência.

Ambas as máquinas foram dedicadas exclusivamente à execução dos experimentos, sem aplicações ou serviços concorrentes em execução, reduzindo interferências no uso de CPU, memória e rede.

A máquina servidora possuía as seguintes características:

- Processador: Apple M2 (8 núcleos)
- Memória RAM: 8 GB
- Sistema Operacional: macOS Tahoe (versão 26.1)

¹ https://github.com/StephanyeCunto/TCC_Java_Thread_Benchmark

- *Java Development Kit* (JDK): versão 21 LTS
- *Java Mission Control* (JMC): versão 9.0
- *Spring Boot*: versão 3.3.2
- Conexão de rede: *Wi-fi* 50 Mbps

A máquina cliente possuía as seguintes especificações:

- Processador: Intel Core i5-2410M (2 núcleos, 4 *threads*)
- Memória RAM: 8 GB
- Sistema Operacional: Ubuntu 22.04 LTS
- Ferramenta de carga: *Vegeta* (versão 12.13.0)
- Conexão de rede: *Wi-fi* 50 Mbps

Todos os experimentos foram realizados com a comunicação entre servidor e cliente por meio de rede *Wi-fi* o que pode introduzir variações adicionais de latência. Entretanto, como ambos os mecanismos de *threads* foram avaliados sob as mesmas condições, esse impacto é considerado equivalente para fins comparativos.

Para evitar que o sistema operacional se tornasse um gargalo durante os experimentos, foram aplicadas configurações específicas em ambas as máquinas. Essas configurações visaram garantir que limitações observadas estivessem associadas ao mecanismo de gerenciamento de *threads*, e não a restrições impostas pelo sistema operacional ou pela pilha de rede.

As seguintes configurações foram realizadas em ambas as máquinas:

- Número máximo global de arquivos abertos no sistema (1048576), evitando que o limite de descritores de arquivo restrinja o número de conexões TCP simultâneas.
- Tamanho máximo da fila de conexões pendentes (4096), permitindo a aceitação de um grande volume de conexões simultâneas.
- Tamanho máximo do *buffer* de *socket* (8 MB), otimizando a transferência de dados pela rede.
- Tamanho do *buffer* de envio e recebimento TCP (2 MB), reduzindo gargalos de comunicação.
- Faixa de portas TCP (1024 a 65.535), evitando o esgotamento de portas efêmeras durante testes intensivos.

Para a máquina servidora também foram realizadas as seguintes configurações:

- Número máximo de arquivos abertos por processo (1048576), permitindo que a aplicação Java mantenha múltiplas conexões simultâneas.
- Número máximo de processos por usuário (2000), viabilizando a criação de múltiplas *threads*.
- Redução do tempo que conexões TCP permanecem em estado `TIME_WAIT` (250 ms), liberando recursos mais rapidamente.
- Desativação do *delayed ACK* do TCP, reduzindo a latência nas comunicações de rede.

Na máquina cliente, por sua vez, também foram aplicadas as seguintes configurações:

- Aumento do número máximo de identificadores de processo (2000000), suportando alta concorrência durante a geração de carga.
- Redução do tempo de finalização de conexões TCP (`FIN_TIMEOUT`), acelerando o encerramento de conexões.
- Ativação do reuso de conexões TCP em estado `TIME_WAIT`, permitindo maior taxa de criação de conexões.

As configurações da JVM foram mantidas em seus valores padrão para refletir cenários reais de uso.

4.3 Implementação do Servidor

Foi desenvolvido um servidor HTTP simples utilizando o *framework Spring Boot*. A escolha do *Spring Boot* se deve à facilidade de uso para a criação rápida de aplicações *WEB* em Java e sua popularidade, sendo listado entre os *frameworks WEB* mais populares em pesquisas, como a pesquisa realizada por Stack Overflow (2025).

O servidor foi implementado de forma a suportar *threads* tradicionais e *threads* virtuais, permitindo a comparação direta entre os dois mecanismos.

Para evitar que o servidor se tornasse um gargalo durante os experimentos, foram aplicadas configurações no *Spring Boot* que permitem alto nível de concorrência:

- `server.tomcat.max-threads=50000`: Define o número máximo de *threads* que o servidor pode criar para processar requisições.

- `server.tomcat.accept-count=50000`: Define o tamanho da fila de conexões pendentes.
- `server.tomcat.connection-timeout=5000`: Define o tempo máximo em milissegundos que o servidor espera por uma conexão.
- `spring.threads.virtual.enabled=true`: Habilita o uso de *threads* virtuais no *Spring Boot*.

O servidor expõe três *endpoints REST*:

- `/threads/traditional`: Processa requisições utilizando *threads* tradicionais do sistema operacional. Cada requisição inicia uma *thread* que aguarda 100 ms antes de responder com *status* HTTP 200.
- `/threads/virtual`: Processa requisições utilizando *threads* virtuais. A lógica de processamento é idêntica ao *endpoint* tradicional, diferindo apenas no mecanismo de *threading* subjacente.
- `/threads/gc`: *Endpoint* auxiliar para executar manualmente o processo de *garbage collection*.

A espera de 100 ms foi escolhida baseada no trabalho de Gustafsson e Persson (2024) para simular operações *I/O-bound* típicas de aplicações *WEB* reais, como consultas a bancos de dados ou chamadas a serviços externos. Esse tempo de bloqueio permite avaliar como cada mecanismo de *threads* lida com alta concorrência em cenários onde *threads* frequentemente aguardam operações de entrada/saída.

4.4 Cenários avaliados

As comparações entre *threads* tradicionais e virtuais consideraram dois cenários de carga distintos: um cenário com solicitação de criação de *threads* de forma constante (cenário 1) e um com aumento gradual desta solicitação (cenário 2). Os dois cenários foram descritos em scripts automatizados baseados no proposto por Gustafsson e Persson (2024), incluindo fases de aquecimento (*warmup*), coleta manual de lixo (*garbage collection*) e pausas entre testes para garantir condições equivalentes.. Adicionalmente, este trabalho propõe alternar a ordem de execução entre *threads* tradicionais e virtuais em cada rodada para minimizar vieses relacionados ao aquecimento da JVM ou estado do sistema.

O número de execuções foi definido com base na tese de Gustafsson e Persson (2024), inicialmente em 5 repetições por cenário, totalizando 10 execuções (5 para *threads* tradicionais e 5 para *threads* virtuais). No entanto, observou-se que 2 a 3 execuções se

configuravam como *outliers*, comprometendo a confiabilidade dos resultados. Para mitigar esse problema, o número de execuções foi aumentado para 50 repetições por cenário, totalizando 100 execuções, aumentando a robustez estatística dos resultados ao considerar a presença de *outliers* e a variabilidade natural das métricas.

Optou-se por 50 execuções por cenário, pois, a partir desse ponto, os ganhos em precisão das métricas tornam-se pequenos, considerando o aumento significativo no tempo de execução e nos recursos computacionais necessários. A Tabela 1 apresenta a estimativa de redução da margem de erro da taxa de requisições (*rate*) e do erro relativo para *threads* tradicionais conforme o número de execuções aumenta.

Tabela 1 – Redução estimada da margem de erro do *rate* (*threads* tradicionais), erro relativo e tempo total de execução.

Execuções	Margem de Erro Estimada (<i>rate</i>)	Erro Relativo (%)	Tempo Total (dias e h)
50	318,45	14,2	6d 4h
60	289,51	12,9	7d 9h
80	249,44	11,1	9d 20h
100	222,43	10,0	12d 7h

Fonte: Elaborado pela autora (2025).

A margem de erro estimada foi calculada a partir da fórmula clássica de intervalo de confiança para a média:

$$ME = t_{\alpha/2, df} \cdot \frac{s}{\sqrt{n}}$$

onde:

- $t_{\alpha/2, df}$ é o valor crítico da distribuição t de *Student* para 95% de confiança e df graus de liberdade;
- s é o desvio padrão da métrica coletada (neste caso, o *rate* das *threads*);
- n é o número de execuções realizadas.

Para 60, 80 e 100 execuções, a margem de erro foi projetada considerando que ela diminui proporcionalmente à raiz quadrada do número de execuções, ou seja:

$$ME_{n_{\text{novo}}} = ME_{50} \cdot \sqrt{\frac{50}{n_{\text{novo}}}}$$

O erro relativo é definido como a razão entre a margem de erro estimada e a média da métrica analisada, expressa em termos percentuais, conforme a Equação 4.1.

$$ER = \frac{ME}{\bar{x}} \times 100 \quad (4.1)$$

onde:

- ER representa o erro relativo, expresso em porcentagem;
- ME é a margem de erro estimada do *throughput*;
- $\bar{x}$ corresponde ao valor médio do *rate* obtido nos experimentos, como referência o valor médio do *throughput* obtido com 50 execuções foi utilizado para todos os valores de execução.

O tempo total de execução foi calculado somando os tempos médios de cada execução e incluindo as pausas entre os testes, convertendo o resultado em dias e horas.

Observa-se que, embora aumentar o número de execuções reduza a incerteza das métricas, o ganho em precisão torna-se pequeno a partir de 50 execuções, enquanto o tempo total necessário cresce significativamente.

4.4.1 Cenário 1: cenário de criação de *threads* de forma constante

O cenário 1 tem por objetivo avaliar a estabilidade e o desempenho dos mecanismos de *threads* sob uma carga constante ao longo do tempo. A carga foi mantida em um nível fixo, permitindo observar como cada mecanismo lida com a concorrência e o consumo de recursos em um cenário estável.

Os parâmetros para os testes estão descritos abaixo e baseiam-se no trabalho de Gustafsson e Persson (2024).

1. Três séries de aquecimento (*warmup*) com 300 requisições por segundo durante 60 segundos.
2. Aquecimento (*warmup*) com requisições constante durante 120 segundos.
3. Pausa de 60 segundos.
4. Coleta de lixo (*garbage collection*) manual na máquina servidor.
5. Pausa de 20 segundos.
6. Teste principal com 1000 requisições por segundo durante 600 segundos.
7. Pausa de 60 segundos.
8. Repetição dos passos 1 a 7 para o outro mecanismo de *threads*, em uma nova instância da JVM.

Neste trabalho, a taxa constante de requisições por segundo foi definida a partir de testes exploratórios preliminares. Nesses testes, a taxa de requisições do script foi incrementada gradualmente até a identificação de um limiar próximo ao surgimento de erros de conexão na máquina servidora. A partir desse limiar, foi selecionado um valor imediatamente inferior, de modo a garantir a estabilidade do ambiente experimental e evitar interferências causadas por falhas de comunicação. Assim, adotou-se a taxa de 1000 requisições por segundo nos experimentos apresentados

4.4.2 Cenário 2: cenário de criação de *threads* com aumento gradual (*ramp up*)

O cenário 2 teve por objetivo desse teste foi identificar o ponto em que o servidor começa a apresentar erros de conexão, indicando que atingiu seu limite máximo de requisições processáveis.

Os parâmetros para os testes estão descritos abaixo e baseiam-se no trabalho de (Gustafsson; Persson, 2024). Neste trabalho, adotou-se o valor identificado no cenário 1 de 1000 requisições por segundo como limiar seguro máximo a ser alcançado nos testes.

1. **Warmup:**

- a) Aquecimento com 300 requisições por segundo durante 60 segundos.
- b) Pausa de 60 segundos.
- c) Coleta de lixo (*garbage collection*) manual na máquina servidor.
- d) Pausa de 20 segundos.
- e) Repetição dos passos acima mais duas vezes, com a taxa ajustada para o limiar seguro de requisições por segundo.

2. **Teste principal:**

- a) Envio de requisições na taxa definida durante 10 segundos.
- b) Pausa de 60 segundos.
- c) Coleta de lixo (*garbage collection*) manual na máquina servidor.
- d) Pausa de 20 segundos.
- e) Aumento da taxa em 50 requisições por segundo.
- f) Repetição dos passos acima até que o servidor comece a apresentar erros de conexão.

3. Pausa de 60 segundos.

4. Repetição dos passos 1 e 2 para o outro mecanismo de *threads*, em uma nova instância da JVM.

O teste foi finalizado quando o servidor apresentou falhas em três execuções consecutivas, indicando saturação consistente do sistema.

4.5 Métricas Coletadas

As métricas foram coletadas em dois pontos: na máquina servidora e na máquina cliente.

No servidor as métricas foram coletadas através do *Java Flight Recorder* (JFR), e processadas com *Java Mission Control* (JMC):

1. Utilização média de CPU (%)
2. Consumo de memória RAM (MB)
3. Utilização de *heap* da JVM (MB)

Os dados foram salvos em arquivos `Monitor.jfr` para cada execução, permitindo análise do comportamento interno da aplicação.

As métricas foram coletadas no cliente pela ferramenta *Vegeta* durante o envio de requisições:

1. Latência: mínima, máxima, média e percentis (P50, P90, P95, P99)
2. Taxa de transferência (*throughput*)
3. Taxa de amostragem
4. Taxa de sucesso (%)
5. Volume de dados (*bytes* de entrada e saída)

Esses dados foram exportados para arquivos JSON para análise e comparação entre os mecanismos de *threads*.

4.6 Tratamento estatísticos dos dados

Os dados apresentados na próxima seção tiveram os *outliers* removidos para as principais métricas CPU, uso de memória e *heap*. Para o teste de carga constante, a métrica analisada foi a latência média, enquanto que, no teste de carga gradual, considerou-se o *rate* máximo.

A detecção e remoção de *outliers* foi realizada utilizando o método dos quartis. Os dados da métrica analisada foram ordenados em ordem crescente. Para um conjunto com n observações, a posição do k -ésimo quartil ($k = 1, 2, 3$) é calculada pela fórmula:

$$P_k = \frac{k(n+1)}{4}$$

onde:

- P_k representa a posição do quartil no conjunto de dados;
- n é o número total de observações;
- $k = 1$ para o primeiro quartil (Q_1), $k = 2$ para a mediana (Q_2) e $k = 3$ para o terceiro quartil (Q_3).

O valor do quartil é obtido da seguinte forma:

- Se P_k for um número inteiro, o quartil corresponde ao valor nessa posição do conjunto ordenado;
- Se P_k for decimal, o quartil é obtido por interpolação linear entre os valores adjacentes.

Com Q_1 e Q_3 definidos, o intervalo interquartil é calculado como:

$$IQR = Q_3 - Q_1$$

e os limites para identificação de *outliers* são:

$$\text{Limite inferior} = Q_1 - k \cdot IQR$$

$$\text{Limite superior} = Q_3 + k \cdot IQR$$

onde k é igual a 1,5. Valores x_i fora do intervalo

$$[Q_1 - k \cdot IQR, Q_3 + k \cdot IQR]$$

foram considerados *outliers* e removidos da análise.

4.7 Tratamento de erros

Durante os experimentos, foram observados erros de conexão e tempo limite, todos os erros ocorreram devido à falta de resposta do servidor, indicando que o mesmo não conseguiu processar todas as requisições recebidas. Esses erros foram identificados através de *tokens* gerado pelo *Vegeta*.

Como exemplo, alguns erros observados:

- EOF (`Client.Timeout exceeded while awaiting headers`)
- http: server closed idle connection (`Client.Timeout exceeded while awaiting headers`)
- context deadline exceeded (`Client.Timeout exceeded while awaiting headers`)

Execuções que apresentaram erros foram mantidas, pois a presença de falhas indica o ponto de saturação do sistema, uma métrica relevante para avaliar escalabilidade.

4.8 Limitações dos Testes

Como já mencionado, os testes possuem algumas limitações que devem ser consideradas na interpretação dos resultados:

- **Limitação de *Hardware*:** Não foi possível atingir os limites absolutos da máquina servidora, uma vez que o sistema operacional impôs restrições no número de requisições simultâneas. Os testes, portanto, caracterizam o limite do sistema operacional, não o limite de *hardware*.
- **Rede *Wi-Fi*:** A utilização de uma conexão *Wi-Fi* pode ter introduzido variabilidade adicionais de latência e perda de pacotes. Embora essa limitação afete ambos os mecanismos igualmente, testes em ambiente cabeado poderiam revelar diferenças mais sutis.
- **Ambiente Controlado:** Os testes foram realizados em um ambiente controlado com carga sintética e podem não refletir completamente o comportamento em ambientes de produção com cargas e padrões de uso variados.
- **Simplificação da Aplicação:** o servidor implementado utiliza apenas operações de espera (*sleep*), não incluindo operações I/O reais como acesso a banco de dados ou chamadas a serviços externos, o que pode limitar a generalização dos resultados.

5 Resultados e Discussão

Esta seção apresenta e discute os resultados obtidos nos experimentos comparativos entre *threads* tradicionais e virtuais. A análise está organizada em dois cenários: carga constante, que avalia a estabilidade e eficiência sob demanda uniforme, e carga gradual (*ramp-up*), que identifica os limites operacionais e o comportamento sob estresse crescente.

5.1 Resultados obtidos referentes ao Cenário 1 - carga constante:

Foram conduzidos experimentos com o objetivo de avaliar a estabilidade e o desempenho dos mecanismos de *threads* sob uma carga constante ao longo do tempo. Isso permitiu observar como cada mecanismo lida com a concorrência e o consumo de recursos em um cenário estável. Os resultados dos experimentos são apresentados nas Tabelas 2 e 3.

Tabela 2 – Média da latência sob carga constante

Endpoint	Lat. Média (s)	P90 (s)	P95 (s)	P99 (s)	Desvio padrão (s)
Virtual	0,1438	0,1398	0,2189	1,0600	0,4370
Tradicional	0,1762	0,1553	0,3346	1,9432	0,6523

Fonte: Elaborado pela autora (2025).

Com base nos dados apresentados na Tabela 2, observa-se que as *threads* virtuais superaram as *threads* tradicionais em todas as métricas de latência avaliadas. A latência média das *threads* virtuais foi de 0,1438 s, enquanto as *threads* tradicionais registraram uma latência média de 0,1762 s, indicando tempos de resposta menores no uso de *threads* virtuais.

Além disso, os percentis P90, P95 e P99 também apresentaram valores inferiores para as *threads* virtuais, o que evidencia não apenas menor latência média, mas também menor variabilidade nos tempos de resposta. Esse comportamento sugere maior estabilidade do sistema quando utiliza *threads* virtuais, especialmente em cenários com múltiplas requisições concorrentes.

A menor latência e a menor variabilidade observadas podem ser atribuídas ao modelo de execução das *threads* virtuais, que são escalonadas pela JVM e multiplexadas sobre um conjunto reduzido de *threads* do sistema operacional. Esse modelo reduz o *overhead* associado à criação, ao escalonamento e ao gerenciamento de *threads* tradicionais, contribuindo para uma resposta mais eficiente às requisições (OpenJDK, 2023).

A taxa de sucesso média manteve-se próxima de 100% para ambos os mecanismos, não sendo observadas diferenças relevantes nesse aspecto. De forma semelhante, o *throughput* permaneceu praticamente constante, variando entre 998 e 999 requisições por segundo

para ambos os modelos de *threads*, indicando que, sob carga constante, ambos são capazes de sustentar níveis semelhantes de vazão.

Tabela 3 – Utilização média de recursos sob carga constante

Endpoint	CPU (%)	Memória (MB)	Heap (MB)
Virtual	$3,70 \pm 0,11$	$410,72 \pm 149,33$	$142,24 \pm 86,77$
Tradicional	$8,26 \pm 0,10$	$462,22 \pm 124,54$	$179,63 \pm 73,73$
Diferença (%)	-55,2%	-11,1%	-20,8%

Fonte: Elaborado pela autora (2025).

Em relação à utilização de recursos computacionais é possível observar na tabela 3 que as *threads* virtuais também apresentam vantagens significativas. A utilização média de CPU foi aproximadamente 55% menor para as *threads* virtuais sendo 3,70%, enquanto as *threads* tradicionais apresentaram um consumo médio de 8,26%. Isso indica que as *threads* virtuais são mais eficientes no uso da CPU, permitindo que o sistema processe mais requisições com menos recursos computacionais.

O consumo médio de memória apresentou redução de aproximadamente 11% para as *threads* virtuais (410,72 MB vs 462,22 MB), enquanto o uso de *heap* da JVM foi cerca de 20% menor (142,24 MB vs 179,63 MB). Embora essas diferenças sejam menos expressivas que a observada na CPU, elas demonstram uma utilização geral mais eficiente dos recursos computacionais.

Um aspecto que merece atenção é o maior desvio padrão no consumo de memória das *threads* virtuais (149,33 MB vs 124,54 MB nas tradicionais). Esse comportamento pode ser atribuído ao modelo de gerenciamento dinâmico de pilhas implementado pela JVM para *threads* virtuais. Conforme documentado pela Oracle (Oracle Corporation, 2025), as pilhas são alocadas e redimensionadas dinamicamente conforme necessário, o que pode resultar em flutuações momentâneas no consumo de memória durante o escalonamento e multiplexação das *threads* virtuais sobre as *threads carrier* do sistema operacional.

Os ganhos observados podem ser explicados pelo modelo arquitetural das *threads* virtuais. Ao contrário das *threads* tradicionais, que mantêm um mapeamento 1:1 com *threads* do sistema operacional, as *threads* virtuais são multiplexadas sobre um conjunto reduzido de *threads carrier* gerenciadas pela JVM (OpenJDK, 2023). Esse modelo reduz significativamente:

1. O *overhead* de criação e destruição de *threads*;
2. O número de chamadas ao sistema (*system calls*);
3. O custo de troca de contexto no nível do sistema operacional;

4. A contenção por recursos do *kernel*.

Apesar das vantagens observadas sob carga constante, é importante analisar o comportamento dos mecanismos de *threads* em cenários que o número de requisições varia ao longo do tempo. Essa análise é apresentada no teste de carga gradual, discutido na seção seguinte.

5.2 Resultados obtidos referentes ao Cenário 1 - carga gradual (*Ramp Up*)

Esse teste foi realizado com o intuito de identificar o ponto em que o servidor começa a apresentar erros de conexão, indicando que atingiu seu limite máximo de requisições processáveis. Os resultados dos experimentos são apresentados nas Tabelas 4 e 5.

Tabela 4 – Taxa média de requisições (*rate*) sob carga gradual

Endpoint	Rate	Desvio Padrão
Virtual	2.072,14	828,42
Tradicional	2.008,16	940,78

Fonte: Elaborado pela autora (2025).

De acordo com os dados apresentados na Tabela 4, observa-se que as *threads* virtuais tiveram uma taxa média de requisições muito próxima as *threads* tradicionais. As *threads* virtuais alcançaram uma taxa média de 2.072,14 requisições por segundo apenas 3,2% superior as *threads* tradicionais que apresentaram uma taxa média de 2.008,16 requisições por segundo. Essa diferença, embora favorável às *threads* virtuais, não é estatisticamente significativa considerando os desvios padrão.

É importante notar que os elevados desvios padrão refletem a natureza do teste: as medições incluem desde taxas iniciais baixas até o ponto de saturação do sistema, naturalmente resultando em alta variabilidade. Esse comportamento era esperado e não compromete a validade das comparações.

Tabela 5 – Utilização média de recursos sob carga gradual

Endpoint	CPU (%)	Memória (MB)	Heap (MB)
Virtual	2,26 ± 0,73	478,40 ± 0,07	188,96 ± 43,32
Tradicional	3,95 ± 1,54	479,08 ± 0,10	195,34 ± 56,36

Fonte: Elaborado pela autora (2025).

Ao observar a Tabela 5, nota-se que a semelhança entre os mecanismos também se reflete no consumo de recursos computacionais. O uso médio de CPU apresenta uma diferença pouco expressiva, com as *threads* virtuais sendo ligeiramente mais eficientes,

registrando aproximadamente 1,69% a menos de utilização em comparação às *threads* tradicionais. Essa diferença, ainda que pequena, indica maior eficiência no uso do processador, especialmente em cenários com grande número de conexões simultâneas. Além disso, os menores valores de desvio padrão observados para as *threads* virtuais sugerem um comportamento mais estável e previsível à medida que a carga aumenta.

O consumo de memória e *heap* apresentou valores praticamente equivalentes entre os mecanismos: 478,40 MB vs 479,08 MB para memória total, e 188,96 MB vs 195,34 MB para *heap*. Esses resultados contrastam com as diferenças observadas no teste de carga constante e sugerem que, próximo ao limite de saturação, o consumo de memória converge independentemente do mecanismo de *threading* utilizado.

Apesar disso, os valores dos desvios padrão sugerem que, assim como observado no *rate* máximo, as *threads* virtuais apresentaram um comportamento mais estável em termos de utilização de CPU, memória e *heap*, enquanto as *threads* tradicionais exibiram maior variabilidade nesses aspectos quando sujeitas a cargas crescentes.

Esse comportamento pode ser explicado pelo fato de que, em cenários de carga extrema, o gargalo do sistema tende a deslocar-se do modelo de concorrência para outros componentes da arquitetura, como a capacidade de CPU, a largura de banda de rede ou as limitações intrínsecas da própria aplicação. No contexto dos experimentos realizados, esse gargalo foi imposto pelo próprio sistema, que passou a limitar o número máximo de conexões suportadas pela máquina servidora, conforme discutido anteriormente. Sendo assim, os benefícios das *threads* virtuais (redução de *overhead* de criação, menor número de *system calls*, e gerenciamento mais eficiente pela JVM) foram mascarados por gargalos. O que explica, apesar da menor utilização de CPU, o *throughput* máximo não apresentar diferença substancial entre os mecanismos.

Semelhante ao trabalho de Gustafsson e Persson (2024), as *threads* virtuais apresentaram menor uso de *heap* e memória, embora com valores próximos aos observados nas *threads* tradicionais. Esse comportamento pode ser explicado pelo modelo de gerenciamento da pilha de execução adotado pelas *threads* virtuais, que utilizam pilhas segmentadas e alocadas dinamicamente no *heap*, em contraste com as pilhas nativas de tamanho fixo associadas às *threads* tradicionais. No entanto, dado o escopo e a configuração dos experimentos realizados, essa diferença estrutural não se refletiu em uma redução expressiva no consumo total de memória.

A latência observada por Gustafsson e Persson (2024) também foi semelhante à verificada neste trabalho, no qual as *threads* virtuais apresentaram melhor desempenho, isto é, menor latência. No entanto, enquanto o uso de CPU observado por Gustafsson e Persson (2024) foi menor para as *threads* tradicionais, tal comportamento difere dos resultados obtidos nos experimentos realizados neste trabalho.

De forma geral, os resultados dos testes demonstram que as *threads* virtuais não apenas alcançam desempenho comparável ao das *threads* tradicionais em termos de taxa máxima de requisições, como também apresentam maior estabilidade no uso de recursos computacionais quando utilizadas em tarefas que demandam alta concorrência e escalabilidade. Essa característica torna o modelo de *threads* virtuais adequado para aplicações que demandam alta escalabilidade e previsibilidade de comportamento sob cargas variáveis.

5.2.1 Implicações Práticas

Os resultados indicam que a escolha entre *threads* virtuais e tradicionais deve considerar o regime operacional da aplicação:

- **Em cargas normais a moderadas** (teste de carga constante): *threads* virtuais oferecem vantagens claras em latência, utilização de CPU e estabilidade, tornando-as preferíveis para a maioria dos cenários de produção.
- **Em cenários de carga crescente** (teste de carga gradual): observou-se que as diferenças de *throughput* máximo entre *threads* virtuais e tradicionais tornam-se poucas expressivas. Nos experimentos realizados, esse comportamento esteve associado a limitações impostas pelo próprio sistema, que passou a atuar como gargalos predominantes. Dessa forma, embora as *threads* virtuais mantenham menor utilização média de CPU e maior estabilidade, os benefícios associados ao modelo de *threading* tendem a ser parcialmente mascarados por limitações externas à JVM.

Em síntese, as *threads* virtuais demonstraram superioridade em eficiência de recursos e latência, em condições operacionais normais. Sob estresse extremo, o desempenho converge devido a gargalos em outras camadas da arquitetura, mas as *threads* virtuais mantêm vantagem em estabilidade e previsibilidade.

6 Conclusão

Este trabalho analisou o desempenho de *threads* tradicionais e *threads* virtuais em aplicações concorrentes Java, com foco em operações *I/O-bound*. Avaliando métricas de latência, *throughput* e utilização de recursos sob regimes de carga constante e gradual, permitindo identificar os cenários em que cada mecanismo apresenta vantagens.

Com base nos resultados obtidos sob carga constante, conclui-se que, as *threads* virtuais apresentaram uma vantagem em relação às *threads* tradicionais, caracterizada por menor latência média e maior estabilidade nos tempos de resposta. Observou-se também que o uso de *threads* virtuais proporcionou melhor eficiência no uso dos recursos computacionais, com menor consumo de CPU, memória e *heap* da JVM. Apesar desses benefícios, verificou-se uma maior variabilidade no consumo de memória quando comparadas às *threads* tradicionais, comportamento associado ao gerenciamento dinâmico das pilhas implementado pela JVM.

Entretanto, no cenário de carga gradual, observou-se que as vantagens das *threads* virtuais tornam-se menos expressivas. Embora o consumo de CPU tenha permanecido mais eficiente quando comparado às *threads* tradicionais, o uso de memória e de *heap* da JVM apresentou valores muito próximos entre os dois mecanismos. Da mesma forma, o número máximo de requisições processadas por segundo, apesar de ligeiramente superior para as *threads* virtuais, mostrou-se equivalente ao observado nas *threads* tradicionais. Esses resultados indicam que, no contexto dos experimentos realizados, os ganhos gerados pelas *threads* virtuais concentram-se principalmente na eficiência do uso da CPU, enquanto métricas como memória, *heap* e *throughput* máximo tendem a convergir. Ressalta-se que esses resultados foram obtidos em função de limitações externas ao modelo de *threading*, como restrições impostas pelo sistema operacional.

Os resultados indicam que, independentemente do cenário analisado, as *threads* virtuais apresentaram menor utilização de CPU quando comparadas às *threads* tradicionais, além de consumo de memória e uso de *heap* iguais ou inferiores. Destaca-se, contudo, que seus benefícios tornam-se mais evidentes no teste de carga constante, no qual foram observadas menores latências, maior estabilidade e melhor eficiência no uso dos recursos computacionais.

É importante ressaltar que este estudo apresenta algumas limitações que devem ser consideradas na interpretação dos resultados, as quais delimitam o escopo da análise realizada. Dentre essas limitações, destacam-se:

- A realização dos experimentos em um ambiente controlado, com carga sintética, o que pode não representar de forma integral o comportamento de aplicações em ambientes de produção.

- A comunicação entre cliente e servidor por meio de rede Wi-Fi, que pode introduzir variações adicionais de latência durante a execução dos testes.
- A utilização de uma aplicação simplificada, baseada em operações de espera para simular cargas do tipo *I/O-bound*, sem a inclusão de acessos reais a bancos de dados ou chamadas a serviços externos.
- As limitações impostas pelo sistema operacional, que delimitam o alcance máximo de desempenho observado durante os experimentos.

Considerando que este trabalho possui um escopo limitado, sugere-se, para trabalhos futuros, a ampliação e aprofundamento dos experimentos realizados. Nesse sentido, destacam-se as seguintes possibilidades:

- A ampliação dos ambientes de teste, tornando-os mais semelhantes a cenários reais de produção, com a inclusão de operações de entrada e saída reais, como acesso a bancos de dados e chamadas a serviços externos.
- A realização de experimentos em ambientes com rede cabeada, o que pode reduzir variações de latência e permitir uma análise mais precisa do impacto do modelo de concorrência.
- A utilização de diferentes sistemas operacionais, possibilitando a obtenção de resultados distintos, especialmente no que se refere aos limites máximos de requisições suportadas pelo sistema.
- A utilização de dados e variáveis reais durante a execução, visando à investigação do impacto do crescimento da pilha de execução no consumo de memória e no desempenho das aplicações.

Essas extensões podem contribuir para uma compreensão mais abrangente do comportamento e das vantagens das *threads* virtuais em diferentes contextos de aplicação.

Referências

- GitHub. *State of the Octoverse 2024*. 2024. GitHub Blog. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-2024>. Acesso em: 21 out. 2025.
- GUSTAFSSON, E.; PERSSON, O. N. *Comparison of Concurrency Technologies in Java*. 2024. Masters Thesis Lund University, Department of Computer Science. Disponível em: <https://lup.lub.lu.se/luur/download?func=downloadFile&recordId=9166685&fileId=9166687>. Acesso em: 6 nov. 2025.
- KEGEL, D. *The C10K Problem*. 2006. Dan Kegel's Networking Performance Resources. Disponível em: <http://www.kegel.com/c10k.html>. Acesso em: 01 jan. 2026.
- OpenJDK. *JEP 444: Virtual Threads*. 2023. Java Enhancement Proposal. Disponível em: <https://openjdk.org/jeps/444>. Acesso em: 27 dez. 2025.
- OpenJDK Project Loom. *State of Loom: Background and Motivation*. 2020. OpenJDK Project Documentation. Disponível em: https://cr.openjdk.org/~rpressler/loom/loom/sol1_part1.html. Acesso em: 01 jan. 2026.
- Oracle Corporation. *One-to-One Multithreading Model*. 2010. Oracle Documentation. Disponível em: <https://docs.oracle.com/cd/E19620-01/805-4031/ch2mt-30/index.html>. Acesso em: 31 dez. 2025.
- Oracle Corporation. *Java Virtual Machine Options*. 2014. Oracle Java SE 8 Documentation. Inclui opções como -Xss. Disponível em: <https://docs.oracle.com/javase/8/docs/technotes/tools/unix/java.html>. Acesso em: 01 jan. 2026.
- Oracle Corporation. *Thread (Java Platform SE 8)*. 2014. Oracle Java Documentation. Disponível em: <https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html>. Acesso em: 31 dez. 2025.
- Oracle Corporation. *Virtual Threads*. 2023. Oracle Documentation. Disponível em: <https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html>. Acesso em: 27 out. 2025.
- Oracle Corporation. *Virtual Threads*. 2025. Oracle Java SE 25 Documentation. Disponível em: <https://docs.oracle.com/en/java/javase/25/core/virtual-threads.html>. Acesso em: 30 dez. 2025.
- PANDITA, V. *Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads*. 2024. Dissertação de Mestrado National College of Ireland, School of Computing. Disponível em: <https://norma.ncirl.ie/8134/>. Acesso em: 6 nov. 2025.
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. 9. ed. Hoboken, NJ: John Wiley & Sons, 2018. ISBN 978-1-118-06333-0.
- SOUTO, H. F. d. *Uma análise comparativa entre threads e green threads no Java*. 2024. Artigo Curso de Ciência da Computação, Universidade Federal de Campina Grande (UFCG). Disponível em: <https://dspace.sti.ufcg.edu.br/bitstream/riufcg/38147/1/HIARLY%20FERNANDES%20DE%20SOUTO-ARTIGO-CI%C3%84NCIA%20DA%20COMPUTA%C3%87%C3%83O-CEEI%20%282024%29.pdf>. Acesso em: 6 nov. 2025.

Stack Overflow. *Stack Overflow Developer Survey 2025: Technology*. 2025. Stack Overflow Blog. Disponível em: <https://survey.stackoverflow.co/2025/technology/>. Acesso em: 27 dez. 2025.

TANENBAUM, A. S.; BOS, H. *Modern Operating Systems*. 4. ed. [S.l.]: Pearson, 2015.

ÁGUAS, D. J. d. O. *Avaliação dos mecanismos de concorrência na API do Java 8*. 2015. Dissertação de Mestrado Instituto Superior de Engenharia do Porto (ISEP). Disponível em: <https://www.proquest.com/openview/dfad25d71a865e5f8f207800736152ed/1?pq-origsite=gscholar&cbl=2026366&diss=y>. Acesso em: 6 nov. 2025.